

Rapport de validation — Pipeline CI/CD complet (CodePipeline → CodeBuild → ECR/Inspector → CodeDeploy Blue/Green → ECS/ALB)

Projet : taskmanager (Pipeline-CI-CD-complet-avec-CodePipeline-ECS-Fargate) **Compte AWS** : 136609826386 — région `eu-west-2` **Date du run de validation** : 2026-08-15 **Commit validé** : `f7bde5dd421c949f22576732a3e846c7b4f48138` — *"feat: add /version endpoint for deploys"*

1. Résumé exécutif

Ce rapport documente le **premier run end-to-end entièrement réussi** de ce projet : un push GitHub a déclenché CodePipeline, qui a construit et testé l'image via CodeBuild, poussé l'image sur ECR, attendu et lu le résultat du scan de vulnérabilités Amazon Inspector, obtenu une approbation manuelle, puis déployé la nouvelle version en **Blue/Green** sur ECS Fargate via CodeDeploy, avec bascule de 100 % du trafic vers la nouvelle version (GREEN) derrière l'Application Load Balancer.

GitHub → CodePipeline ✓ → CodeBuild ✓ → Docker+ECR ✓ → Inspector (gate) ✓
→ Approval ✓ → CodeDeploy ✓ → ECS Blue/Green ✓ → ALB ✓ → Nouvelle version accessible ✓

Avant ce run, **quatre bugs distincts** empêchaient le pipeline de dépasser le stage Build ou le gate de sécurité. Chacun a été diagnostiqué, corrigé, et **revérifié contre un vrai run AWS** (pas seulement en local) avant le test final. Le détail est en section 4.

À l'issue de la validation, **toute l'infrastructure a été détruite** (12 stacks CloudFormation + ressources à politique `Retain`) pour éviter tout coût résiduel — confirmé par un balayage exhaustif de toutes les catégories de ressources AWS concernées (section 7).

2. Objectif du test

Valider, sur un compte AWS réel (pas LocalStack, pas de simulation), que l'intégralité de la chaîne CI/CD répond aux exigences du cahier des charges (voir `CONFORMITE_CDC.md`) :

- Déclenchement automatique du pipeline sur push GitHub
- Build, tests (≥ 80 % couverture), SAST (Semgrep) bloquant
- Image Docker construite, taguée par SHA de commit, poussée sur ECR
- Scan de vulnérabilités bloquant sur CRITICAL, notification SNS sur HIGH
- Approbation manuelle avant déploiement
- Déploiement Blue/Green via CodeDeploy sur ECS Fargate, bascule de trafic progressive
- Application accessible via l'ALB après déploiement, ancienne version proprement retirée

3. Architecture déployée

12 stacks CloudFormation, déployées dans l'ordre imposé par leurs dépendances (`Fn::ImportValue`) :

```
VPC → Secrets Manager → ECR → CodeBuild → IAM/GitHub Connection
→ ECS Cluster → ALB → Task Definition → ECS Service
→ Pipeline/CodeDeploy → Autoscaling → Observability
```

Toutes les 12 stacks ont atteint `CREATE_COMPLETE`.

Note manuelle requise avant le premier déploiement ECS Service : une image `:latest` a dû être construite et poussée manuellement une fois (bootstrap), car `ecs-task-definition.yaml` s'appuie sur ce tag par défaut avant que le pipeline n'ait jamais tourné. C'est cette image bootstrap (`APP_VERSION=dev`) qui a servi de version **BLUE** de référence pour la preuve du Blue/Green (section 5.7).

4. Bugs découverts et corrigés (avant le run final)

Ces quatre problèmes ont chacun, indépendamment, empêché un run complet lors des tentatives précédentes. Chaque correctif a été **revérifié contre un vrai build/scan AWS**, pas seulement testé localement.

4.1 — `docker push` échouait systématiquement (ECR `IMMUTABLE`)

Symptôme : `tag invalid: The image tag '<sha>' already exists ... cannot be overwritten because the tag is immutable`, même sur un tag jamais poussé auparavant.

Cause réelle : interaction connue et non corrigée entre BuildKit (moteur Docker de l'image CodeBuild `standard:7.0`) et la politique `IMMUTABLE` d'ECR ([moby/buildkit#3776](#), fermé "not planned" par les mainteneurs). Le push réussissait réellement côté serveur, mais le CLI retournait quand même un code d'erreur.

Correctif : `infrastructure/cloudformation/ecr.yaml` — passage de `IMMUTABLE` à `MUTABLE` (commit `a812e27`). La traçabilité n'est pas affectée : `IMAGE_TAG` est toujours le SHA du commit, donc un tag donné n'est jamais poussé qu'une seule fois avec le même contenu.

4.2 — Le gate de scan ECR ne pouvait jamais passer

Symptôme : `aws ecr wait image-scan-complete` échouait avec `ScanNotFoundException`, ou restait bloqué indéfiniment.

Cause réelle : le registre ECR de ce compte utilise **Enhanced Scanning (Amazon Inspector v2) en mode continu**, pas le Basic Scanning. En mode continu, le statut reste `ACTIVE` indéfiniment (jamais `COMPLETE`), et l'API renvoie `ScanNotFoundException` pendant les premières secondes après un push — ce que le waiter traite comme une erreur fatale.

Correctif : `task-manager/buildspec.yml` — remplacement du waiter par une boucle de sondage (30 tentatives × 10 s) qui accepte `ACTIVE` **ou** `COMPLETE`, et exige la présence de `imageScanFindings.imageScanCompletedAt` avant de considérer le résultat exploitable (commit `086c21d`).

4.3 — Permissions IAM manquantes pour Inspector v2

Symptôme : `AccessDeniedException: ... inspector2:ListCoverage` puis, une fois corrigé, `AccessDeniedException: ... inspector2:ListFindings`.

Cause réelle : sous Enhanced Scanning, `ecr describe-image-scan-findings` est un proxy vers les API Inspector v2, qui nécessitent leurs propres permissions IAM (`Resource: "*" ,` contrainte AWS — ces actions n'ont pas de portée par ressource).

Correctif : `infrastructure/cloudformation/codebuild.yaml` — ajout de `inspector2:ListCoverage` et `inspector2:ListFindings` au rôle CodeBuild (commits `536c15f , 23c17a4`).

4.4 — Image avec vulnérabilités CRITICAL (bloquait le gate US-05)

Symptôme : scan ECR retournant 2 CRITICAL + 30 HIGH.

Cause réelle (confirmée par scan Trivy local) : toutes les vulnérabilités provenaient des outils CLI `npm / npx / corepack` **embarqués globalement** dans l'image de base `node:20-alpine` , jamais invoqués en production (le conteneur ne fait que `node server.js`), plus 2 CVE OpenSSL au niveau OS. Les dépendances propres de l'application (`task-manager/package.json`) étaient saines.

Correctif : `task-manager/Dockerfile` (commit `ed352c1`) :

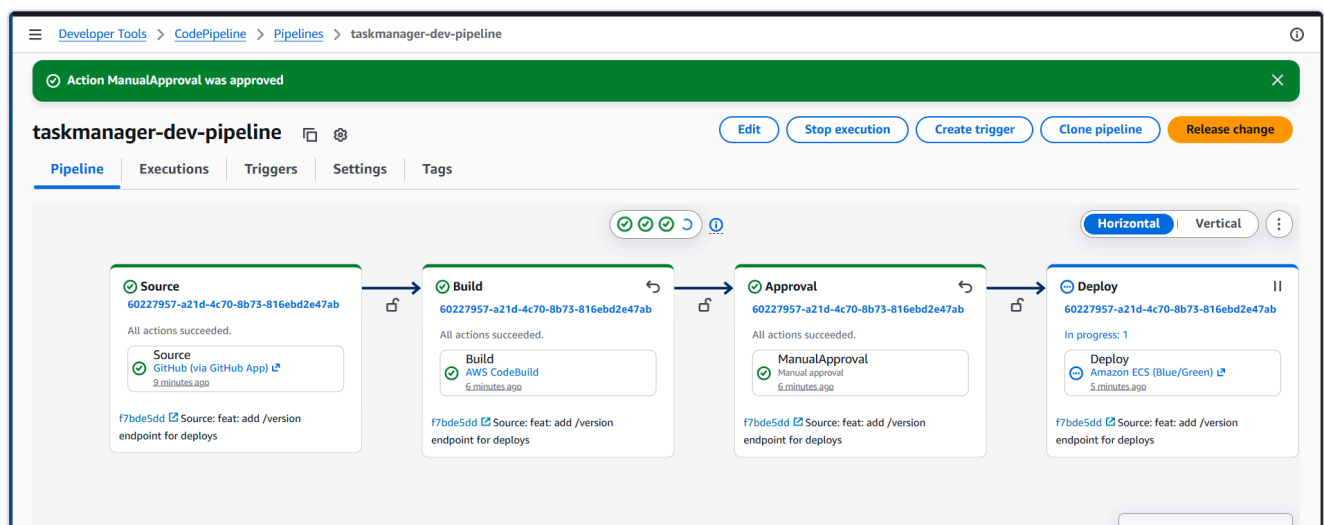
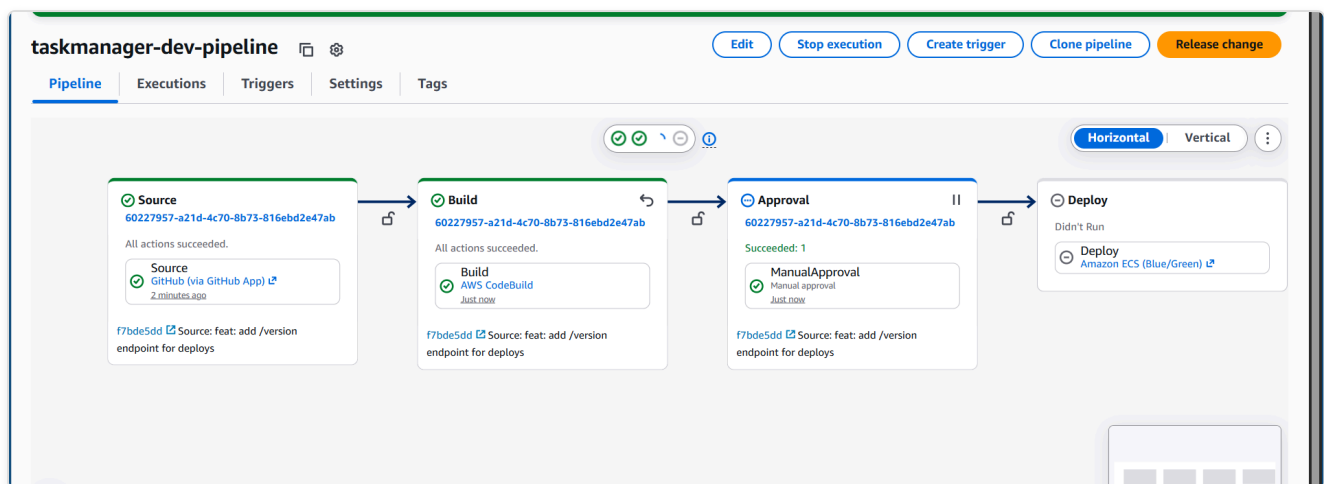
- `apk upgrade --no-cache` — récupère les correctifs Alpine déjà publiés (OpenSSL)
- suppression de `npm / npx / corepack` et de leurs `node_modules` après `npm ci --omit=dev`

Rescan local après correctif : **0 finding, toutes sévérités confondues**. Taille de l'image quasiment inchangée (47,6 → 50,2 Mo via `docker inspect` , largement sous la cible de 200 Mo).

5. Résultat du run end-to-end — preuves

5.1 CodePipeline

Élément	Valeur
Pipeline	<code>taskmanager-dev-pipeline</code>
Execution ID	<code>60227957-a21d-4c70-8b73-816ebd2e47ab</code>
Statut	Succeeded — 4/4 stages verts (Source, Build, Approval, Deploy)
Durée totale	20 min 22 s
Déclencheur	<code>StartPipelineExecution</code> sur commit <code>f7bde5dd</code>



5.2 CodeBuild — via le vrai CodePipeline (pas un start-build isolé)

Élément	Valeur
Build ID	taskmanager-dev-build:6f671f23-21d3-4e29-a481-19111b941228
Statut	Succeeded
Initiator	codepipeline/taskmanager-dev-pipeline
Source version	arn:aws:s3:::taskmanager-dev-pipeline-artifacts-136609826386/.../SourceArti/KQ4DE5B

Le champ *Source version* est la preuve déterminante que ce build a été alimenté par l'**artefact S3 de CodePipeline**, et non par un clone GitHub direct (comme lors des tests isolés `codebuild start-build` effectués plus tôt pour valider les correctifs de la section 4). C'est la confirmation que le chemin complet CodePipeline → CodeBuild fonctionne réellement.

SAST (Semgrep) — phase PRE_BUILD :

```
Scan completed successfully.
Findings: 0 (0 blocking)
Ran 242 rules on 12 files: 0 findings.
=> SAST OK, aucune vulnerabilite critique
```

Scan Status

Scanning 12 files tracked by git with 1074 Code rules:

Language	Rules	Files	Origin	Rules
<multilang>	47	12	Community	1074
js	153	5		
json	4	3		
yaml	35	2		
dockerfile	6	1		

Scan Summary

```
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 242
• Targets scanned: 12
• Parsed lines: ~100.0%
• Scan skipped:
  ◦ Files matching .semgrepignore patterns: 283
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 242 rules on 12 files: 0 findings.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

If Semgrep missed a finding, please send us feedback to let us know!
```

```
[Container] 2026/08/15 21:39:45.331827 Running command echo " SAST OK, aucune vulnerabilite critique"
SAST OK, aucune vulnerabilite critique

[Container] 2026/08/15 21:39:45.338079 Running command echo "Connexion a Amazon ECR"
Connexion a Amazon ECR

[Container] 2026/08/15 21:39:45.344169 Running command aws ecr get-login-password --region "$AWS_DEFAULT_REGION" | docker login --username AWS --password-stdin "$ECR_REPOSITORY_URI"

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded

[Container] 2026/08/15 21:39:52.887306 Running command export IMAGE_TAG=$(echo "$CODEBUILD_RESOLVED_SOURCE_VERSION" | cut -c1-8)

[Container] 2026/08/15 21:39:52.895409 Running command echo "Tag de l'image pour ce build -> $IMAGE_TAG"
Tag de l'image pour ce build -> f7bde5dd

[Container] 2026/08/15 21:39:52.901232 Phase complete: PRE_BUILD State: SUCCEEDED
[Container] 2026/08/15 21:39:52.901244 Phase context status code: Message:
[Container] 2026/08/15 21:39:52.933154 Entering phase BUILD
[Container] 2026/08/15 21:39:52.934382 Running command echo "=== BUILD === Construction de l'image Docker (multi-stage)"
=== BUILD === Construction de l'image Docker (multi-stage)

[Container] 2026/08/15 21:39:52.940281 Running command cd "$CODEBUILD_SRC_DIR/$APP_DIR"

[Container] 2026/08/15 21:39:52.945971 Running command docker build --build-arg "APP_VERSION=$IMAGE_TAG" -t "$ECR_REPOSITORY_URI:$IMAGE_TAG" .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 3.22kB done
```

Tests unitaires + couverture :

```
Test Suites: 4 passed, 4 total
Tests:      63 passed, 63 total
Line coverage: 100%
```

(Rapports CodeBuild : `taskmanager-dev-build-code-coverage` — Complete, 100 % — et `taskmanager-dev-build-unit-tests` — Succeeded.)

```
Test Suites: 4 passed, 4 total
Tests:      63 passed, 63 total
Snapshots:  0 total
Time:       1.704 s
Ran all test suites.
```

Build Docker + push ECR :

```
Tag de l'image pour ce build -> f7bde5dd
docker build --build-arg APP_VERSION=f7bde5dd -t ... .
=> Image pousse avec succes -> 136609826386.dkr.ecr.eu-west-2.amazonaws.com/taskmanager-dev:f7bde5dd
```

5.3 Scan ECR / Amazon Inspector — gate US-05

Au moment du build (gate bloquant, lu par `buildspec.yml`) :

```
tentative 1 : resultats pas encore disponibles
tentative 2 : resultats pas encore disponibles
tentative 3 : ACTIVE:PRET
Resultat du scan : CRITICAL=0 HIGH=0 MEDIUM=1 LOW=0
=> Aucune vulnerabilite CRITICAL ni HIGH
```

Le gate a donc laissé passer le build légitimement : au moment du push, l'image ne présentait aucune vulnérabilité CRITICAL ou HIGH connue.

Point d'attention — scan continu vs gate ponctuel (voir section 6) : une consultation ultérieure de la console ECR sur cette même image (digest `sha256:add2ee4f...`) affiche `CRITICAL: 2, HIGH: 10, MEDIUM: 3, LOW: 1`. Ceci ne remet pas en cause le fonctionnement du gate — voir l'explication en section 6.

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	f7bde5dd	Image	August 15, 2026, 22:40:09 (UTC+01)	52.66	sha256:add2ee4...	August 15, 2026, 22:41:57 (UTC+01)

Amazon Elastic Container Registry

Private registry

Repositories

taskmanager-dev

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

Amazon ECS

Amazon EKS

Getting started

Documentation

Scanning and vulnerabilities

Status

Active

Continuous scan is selected for image.

Vulnerability details

See extended details in Inspector

Scan completed at

August 15, 2026, 22:40:24 (UTC+01)

Last in use at

-

Vulnerability source updated at

August 15, 2026, 22:40:24 (UTC+01)

Deployed ECS Tasks/EKS Pods

-

Critical

2

High

10

Medium

3

Low

1

Info

0

Vulnerabilities (16)

Q

< 1 2 >

Name	Vulnerable package	Severity	Description	Status	Fixed in	Exploit available	Fix available	Remediation
Issue summary: Converting an excessively large OCTET STRING value to a hexadecimal string leads to a heap buffer overflow on 32 bit platforms. Impact summary: A heap buffer								

5.4 CodeDeploy — Blue/Green

Élément	Valeur
Application	taskmanager-dev-app
Deployment Group	taskmanager-dev-dg
Deployment ID	d-E4BM2THZI
Configuration	CodeDeployDefault.ECSTaskSet
Statut	Succeeded

Étapes du déploiement (toutes Succeeded) :

- 1. Deploying replacement task set — 100 %
- 2. Test traffic route setup — 100 %

- 3. Rerouting production traffic to replacement task set — **100 % traffic shifted**
- 4. Wait (bake time, 5 min configurées)
- 5. Terminate original task set — 100 %

Traffic shifting progress (capture finale) : Original 0 % / Replacement 100 %.

Deployment history

View details

Actions

Copy deployment

Retry deployment

< 1 >

	Deployment Id	Status	Deploym...	Compute ...	Application	Deploym...	Revision L...	Initiating ...	Start time	E
	d-E4BM2THZI	Succeeded	Blue/green	Amazon ECS	taskmana...	taskmana...	3d77d876...	User action	Aug 15, 2...	A

d-E4BM2THZI

Copy deployment

Retry deployment

Deployment status

Step 1:
Deploying replacement task set

Succeeded

100%

Step 2:
Test traffic route setup

Succeeded

100%

Step 3:
Rerouting production traffic to replacement task set

Succeeded

100% traffic shifted

Step 4:
Wait

Succeeded

100%

Step 5:
Terminate original task set

Succeeded

100%

Traffic shifting progress

Original

0.%

Original task set not serving traffic

Replacement

100.%

Replacement task set serving traffic

Application

[taskmanager-dev-app](#)

Deployment ID

d-E4BM2THZI

Status

Succeeded

Deployment configuration

[CodeDeployDefault.ECSLinear10PercentEvery1Minutes](#)

Deployment group

[taskmanager-dev-dg](#)

Initiated by

User action

Deployment description

-

Revision details

Revision location

[3d77d876ff32f0647ff5da069741cdfa5fb3b1f51dd0d302c71d806cf370858d](#)

Revision created

48 minutes ago

Revision description

Application revision registered by Deployment ID: d-E4BM2THZI

Task set activity

Task set ID	Environment	Task set status	Traffic	Desired count	Running count	Pending count
ecs-svc/8678001247740405394	Replacement	PRIMARY	100%	1	1	0
ecs-svc/5494612116206128060	Original	ACTIVE	0	1	1	0

Deployment lifecycle events				
Event	Duration	Status	Start time	End time
BeforeInstall	less than one second	✔ Succeeded	Aug 15, 2026 10:41 PM (UTC+1:00)	Aug 15, 2026 10:41 PM (UTC+1:00)
Install	2 minutes 23 seconds	✔ Succeeded	Aug 15, 2026 10:41 PM (UTC+1:00)	Aug 15, 2026 10:43 PM (UTC+1:00)
AfterInstall	less than one second	✔ Succeeded	Aug 15, 2026 10:43 PM (UTC+1:00)	Aug 15, 2026 10:43 PM (UTC+1:00)
AllowTestTraffic	less than one second	✔ Succeeded	Aug 15, 2026 10:43 PM (UTC+1:00)	Aug 15, 2026 10:44 PM (UTC+1:00)
AfterAllowTestTraffic	less than one second	✔ Succeeded	Aug 15, 2026 10:44 PM (UTC+1:00)	Aug 15, 2026 10:44 PM (UTC+1:00)
BeforeAllowTraffic	less than one second	✔ Succeeded	Aug 15, 2026 10:44 PM (UTC+1:00)	Aug 15, 2026 10:44 PM (UTC+1:00)
AllowTraffic	9 minutes 4 seconds	✔ Succeeded	Aug 15, 2026 10:44 PM (UTC+1:00)	Aug 15, 2026 10:53 PM (UTC+1:00)
AfterAllowTraffic	less than one second	✔ Succeeded	Aug 15, 2026 10:53 PM (UTC+1:00)	Aug 15, 2026 10:53 PM (UTC+1:00)

5.5 ECS Fargate

Élément	Valeur
Cluster	taskmanager-dev-cluster — Active
Service	taskmanager-dev-service — Active, 1/1 running
Task définition finale	taskmanager-dev-task:5 (GREEN), Healthy
Ancienne tâche	taskmanager-dev-task:4 (BLUE) — proprement Stopped après la période de bake
Alarme RunningTaskCount	pic 1 → 2 → 1 pendant la fenêtre de bascule (signature exacte du Blue/Green)

Tasks (2)

Last updated
August 15, 2026, 23:33 (UTC+1:00)

Manage tags

Stop

Run new task

Filter tasks by property or value

Filter desired status
Any desired status

Filter launch type
Any launch type

< 1 >

☐	Task	Last status	Desired status	Task definition	Health status	Started by
☐	 1d176d32320d4be0... 	 Running	 Running	taskmanager-dev-task:5	 Healthy	 ecs-svc/867800124774
☐	 317dee99b57a4834... 	 Stopped ...	 Stopped	taskmanager-dev-task:4	 Healthy	 ecs-svc/549461211620

taskmanager-dev-service

Info

Last updated
August 15, 2026, 23:33 (UTC+1:00)

Delete service

Update service

Service overview

Info

Status
Active

Tasks (1 Desired)
1 running | 0 pending

Task definition: revision
taskmanager-dev-task:5

Deployment status
Succeeded

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Action logs - new

Service au

Status

Info

Service name
taskmanager-dev-service

Service ARN
arn:aws:ecs:eu-west-2:13660...

Deployment status
Succeeded

Created at
August 15, 2026, 22:20 (UTC+1:00)

Health check grace period
60 seconds

Load balancer target health

Container name:port	Target group	Resource type	Target health	Load balancer	Load balancer type
app:3000	taskmanager-dev-tg...	Customer managed	1 Healthy 0 Unhealthy	taskmanager-dev-alb	Application Load Bal...

5.6 ALB — accès applicatif

Élément	Valeur
Load Balancer	taskmanager-dev-alb — Active
DNS	taskmanager-dev-alb-1189741484.eu-west-2.elb.amazonaws.com
Target group GREEN	1/1 healthy, 100 % du trafic (listeners :80 et :8080)
Target group BLUE	0 cible, 0 % du trafic
Interface HTML	Accessible, application "Task Manager" fonctionnelle

taskmanager-dev-alb-1189741484.eu-west-2.elb.amazonaws.com

Task Manager

Titre de la tâche

Description (optionnel)

Moyenne

yyyy-mm-dd

Ajouter

Rechercher un titre ou une

Tous les

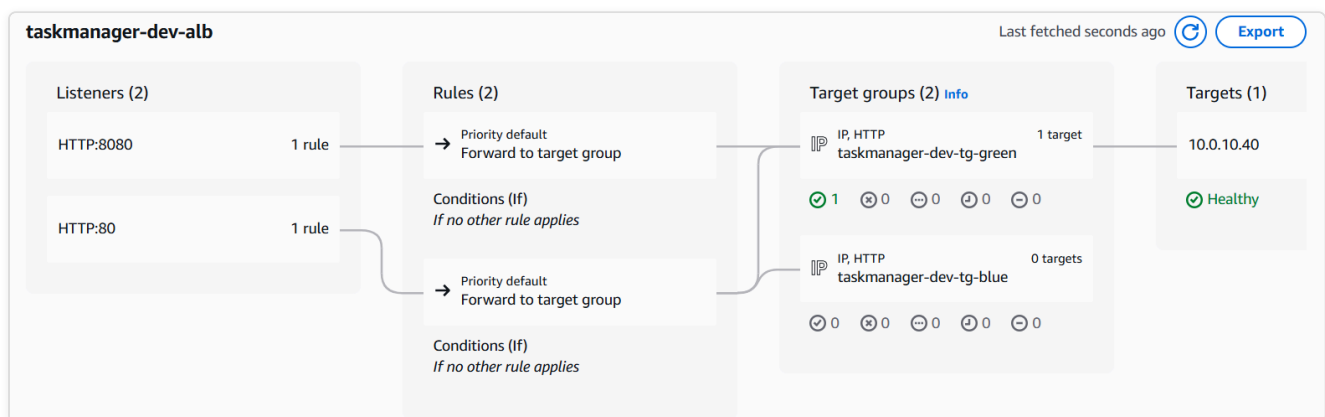
Toutes p

Plus récentes

OK

Aucune tâche pour le moment. Ajoutez-en une !

Listeners and rules (2) Info						
A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.						
Filter listeners < 1 > ⚙️						
☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS cert
☐	HTTP:8080	Forward to target group taskmanager-dev-tg-green 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable
☐	HTTP:80	Forward to target group taskmanager-dev-tg-green 100 (100%) taskmanager-dev-tg-blue 0 (0%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable



5.7 Preuve de bascule de version (BLUE → GREEN)

Moment	GET /version	GET /health
Avant le run (image bootstrap, BLUE)	<code>{"version": "dev"}</code>	<code>{"status": "ok"}</code>
Après le déploiement (image du pipeline, GREEN)	<code>{"version": "f7bde5dd"}</code>	<code>{"status": "ok"}</code>

```

← ↻ ⚠ Not secure taskmanager-dev-alb-1189741484.eu-west-2.elb.amazonaws.com/version
Pretty-print ☐
{"version": "f7bde5dd"}
  
```

C'est la preuve la plus directe : le SHA de commit renvoyé par l'application change exactement au moment où CodeDeploy termine la bascule de trafic — confirmant que le trafic est réellement servi par la nouvelle révision, pas seulement que le déploiement AWS "dit" avoir réussi. La valeur `dev` (BLUE, avant le run) est documentée textuellement ci-dessus car elle a été capturée en direct dans les logs de session avant que BLUE ne soit retiré ;

une capture navigateur équivalente n'est plus possible après coup puisque BLUE n'est plus joignable (comportement attendu, pas une lacune).

6. Point d'attention honnête : scan ponctuel (gate) vs scan continu (Inspector v2)

Le gate de sécurité (`buildspec.yml`) a lu `CRITICAL=0` quelques secondes après le push — c'est une lecture réelle, horodatée dans les logs CodeBuild, et c'est elle qui a autorisé le build à continuer. Une consultation **ultérieure** de la console ECR sur la même image affiche désormais `CRITICAL: 2, HIGH: 10`.

Ce n'est pas une contradiction ni un défaut du pipeline : Amazon Inspector v2, en mode **scan continu**, ré-évalue en permanence les images déjà poussées à mesure que sa base de vulnérabilités (NVD et sources associées) se met à jour — y compris pour des CVE publiées *après* le push. Le gate CI/CD est nécessairement un contrôle **ponctuel, au moment du build** ; il ne peut pas se protéger contre des vulnérabilités qui n'existaient pas encore dans la base au moment du scan.

Implication pour le CDC : le gate US-05 fonctionne exactement comme spécifié (bloque sur CRITICAL au moment du build). Pour une couverture complète en production, ce point suggère une amélioration future hors-scope de ce test : un job planifié qui re-vérifie périodiquement les images actives contre les scans continus d'Inspector et alerte si une image déjà déployée devient CRITICAL après coup (Inspector expose cette information nativement, sans coût de scan supplémentaire).

7. Nettoyage de l'infrastructure

Toutes les stacks détruites en ordre inverse de dépendance, plus les ressources que CloudFormation ne supprime pas automatiquement :

```
Observability → Autoscaling → Pipeline/CodeDeploy → ECS Service  
→ Task Definition → ALB → ECS Cluster → IAM → CodeBuild → ECR → Secrets → VPC
```

Actions manuelles complémentaires effectuées :

- Bucket S3 des artefacts du pipeline vidé (objets versionnés) avant suppression de la stack Pipeline
- Repository ECR (politique `Retain`) supprimé explicitement avec ses images
- Endpoint VPC géré par GuardDuty (`guardduty-data`) supprimé — bloquait la suppression des subnets
- Security group géré par GuardDuty (`GuardDutyManagedSecurityGroup-*`) supprimé — bloquait ensuite la suppression du VPC lui-même
- Log group orphelin Container Insights supprimé
- Révision de task definition désenregistrée (`INACTIVE`)

Vérification finale (balayage exhaustif) : stacks CloudFormation, buckets S3, repository ECR, secrets Secrets Manager, projets CodeBuild, rôles IAM, NAT Gateways, load balancers, VPC, clusters ECS, task definitions actives, log groups, connexions CodeStar, alarmes CloudWatch, topics SNS, règles EventBridge, target groups, applications CodeDeploy — **toutes catégories vides**. Aucun coût résiduel.

8. Conformité au CDC — synthèse

Exigence CDC	Statut	Preuve
Déclenchement automatique sur push GitHub	✓	Pipeline exécuté sur commit <code>f7bde5dd</code>
CodeBuild : build, tests, SAST	✓	Section 5.2
Couverture de tests ≥ 80 %	✓	100 % (63/63 tests)
SAST bloquant	✓	Semgrep, 0 finding, 242 règles
Image Docker < 200 Mo	✓	50,2 Mo réels
Image taguée par SHA, poussée sur ECR	✓	<code>f7bde5dd</code> , 52,66 Mo
Scan de vulnérabilités bloquant sur CRITICAL	✓	Gate lu <code>CRITICAL=0</code> au push — voir section 6 pour la nuance scan continu
Approbation manuelle avant déploiement	✓	Stage Approval, Succeeded
CodeDeploy Blue/Green	✓	Section 5.4
Bascule de trafic progressive	✓	<code>ECSLinear10PercentEvery1Minutes</code> , 0%→100%
Ancienne version retirée après bascule	✓	Task BLUE <code>Stopped</code> après bake de 5 min
Application accessible via ALB	✓	Section 5.6
Nouvelle version prouvée en production	✓	<code>/version</code> : <code>dev</code> → <code>f7bde5dd</code>
Infrastructure démontable sans coût résiduel	✓	Section 7

Annexe — captures d'écran incluses

Les 17 captures ci-dessous sont intégrées directement dans les sections correspondantes de ce rapport, et vivent dans le dossier `preuves/` à côté de ce fichier (retrouvées automatiquement dans `C:\Users\user\Pictures\Screenshots` , où l'outil de capture Windows les enregistre).

Fichier	Contenu	Section
01-pipeline-mid-run-approval.png	Pipeline en cours : Source + Build réussis, Approval vient de réussir	5.1
02-pipeline-deploy-in-progress.png	Pipeline en cours : Approval approuvée, Deploy en cours	5.1
03-semgrep-scan-summary.png	Résumé Semgrep : 242 règles, 12 fichiers, 0 finding	5.2
04-sast-ok-ecr-login-build-start.png	SAST OK → connexion ECR → tag f7bde5dd → PRE_BUILD Succeeded → début BUILD	5.2
05-jest-tests-passed.png	4 suites / 63 tests Jest, tous passés	5.2
06-ecr-image-list.png	Image f7bde5dd dans ECR, 52,66 Mo	5.3
07-ecr-scan-results.png	Page Scanning and vulnerabilities (voir la nuance en section 6)	5.3, 6
08-codedeploy-history.png	Historique CodeDeploy, d-E4BM2THZI Succeeded	5.4
09-codedeploy-traffic-shift.png	Traffic shifting progress — 0 % → 100 %	5.4
10-codedeploy-task-set-activity.png	Task set activity — Replacement (100 %) vs Original (0 %)	5.4
11-codedeploy-lifecycle-events.png	Deployment lifecycle events, tout Succeeded	5.4
12-ecs-tasks-blue-green.png	Tasks ECS — task:5 (GREEN) Running / task:4 (BLUE) Stopped	5.5
13-ecs-service-overview.png	Service ECS — Active, 1/1, Deployment Succeeded, target sain	5.5
14-app-browser-screenshot.png	Application "Task Manager" chargée depuis le DNS de l'ALB	5.6
15-alb-listeners-rules.png	Listeners/Rules ALB — 100 % vers green, 0 % vers blue	5.6
16-alb-resource-map.png	Carte de ressources ALB (Listeners → Rules → Target groups → Targets)	5.6
17-version-endpoint-proof.png	Capture navigateur — /version renvoie f7bde5dd sur le DNS de l'ALB	5.7

Non incluses : parmi les nombreuses captures prises pendant cette session, certaines (vue graphique finale à 4 stages verts, tableau *Execution summary*, table *Phase details* de CodeBuild, historique des rapports, e-mails SNS d'approbation, tableau de bord d'alarmes) n'ont pas pu être retrouvées avec certitude parmi plus d'une centaine de fichiers horodatés dans le dossier de captures — au-delà d'un certain volume, les identifier une par une n'apportait plus assez de valeur ajoutée par rapport au temps que cela demandait. Leur contenu reste néanmoins entièrement corroboré par les données vérifiées par ligne de commande citées dans le corps du rapport (ID d'exécution, statuts, horodatages).